Ch 05: Introduction to Functions

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Introduction to Functions (1 of 2)

- **Function**: group of statements within a program that perform a specific task
  - Usually one task of a large program
    - Functions can be executed in order to perform overall program task
  - Known as **divide and conquer**
  - Known as **functional decomposition** of a problem into subproblems
- **Modularized program**: program wherein each task within the program is in its own function
  - **Code reuse** can reuse functions, can organize functions into modules for reuse.
  - Why Use Functions in Programming (Developer, 2021)
  - How to Write Good Functions (Peppoloni, 2018)

Function

A function is a group of statements that exist within a program for the purpose of performing a **specific** task.



Introduction to Functions (2 of 2)

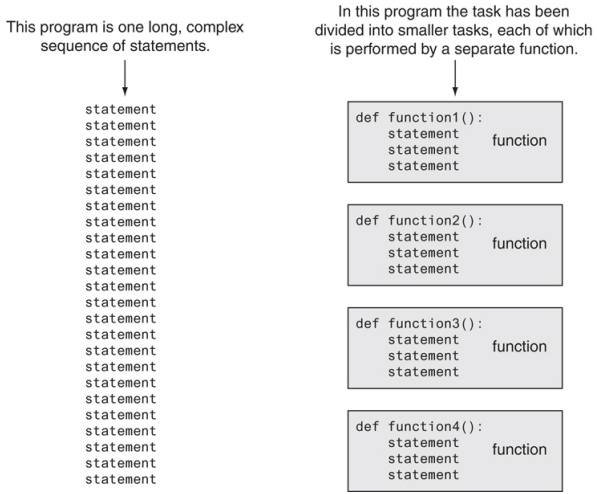


Figure 1: Using functions to divide and conquer a large task (Gaddis, 2021, pg.242)

Benefits of Modularizing a Program with Functions

The benefits of using functions include:

- **Simpler code**
  - Improves readability, high level concepts captured as a function
- **Code reuse**
  - Write the code once and call it multiple times
- **Better testing and debugging**
  - Can test and debug each function individually (e.g. assignment unit tests)
- **Faster development**
  - A side effect of code-reuse, don't have to reinvent the wheel, write once use many
- **Easier facilitation of teamwork**
  - Decomposing into functions naturally allows team of developers to implement different subsets of needed functions for whole program



Void Functions vs. Value-Returning Functions

- A **void function**:
  - Simply executes the statements it contains and then terminates.
  - e.g. no `return` statement, Python `print()` function
- A **value-returning function**:
  - Executes the statements it contains, and then it returns a value back to the statement that called it
  - The `input()`, `int()` and `float()` functions are examples you have used



Defining and Calling a Function (1 of 2)

- Need 3 things to define and use a function
  1. Need to name the function (naming rules same as for variables)
     - Function name should be description of the task
     - Often includes a verb
     - Creating Meaningful Names (Pacheco, 2023)
  2. Need to determine the input parameters that will be passed into the function
  3. Need to determine if the function returns a value, and if so what value(s)

Function definition: specifies what function does when called

```
def function_name(parameter1, parameter2):
    statement
    statement
    return value # if value returning function
```



Defining and Calling a Function (2 of N)

- Call a function to execute it
- When a function is called:
  - Interpreter jumps to the function and executes statements in the function block
  - Interpreter jumps back to part of program that called the function when done
    - Known as function return

```
# define the function,
# no input parameters,
# no return result
def message():
    print('I am Arthur,')
    print('King of the Britons.')

# call the function
message()
```



Indentation in Python

- Each block of statements must be indented (Python syntax to define a code block)
  - Lines in block must begin with the same number of spaces
    - Use spaces to indent lines in block, never create a file that has embedded tabs, or mixed tabs and spaces for Python indentation Python PEP8 Style Guide
    - IDLE and most IDE for Python automatically indents the links in a block
  - Blank lines that appear in a block are ignored
  - Line comments in a block should be indented same as code in the block



EAST TEXAS A&M

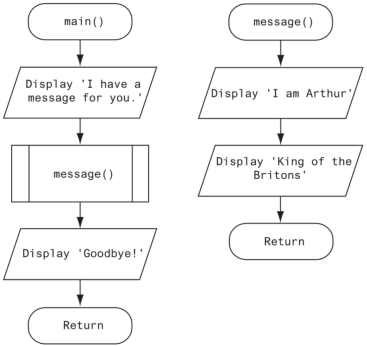
Designing a Program to Use Functions

- **Top-down design**: technique for breaking algorithm into functions

Top-Down Design

Programmers commonly use a technique known as top-down design to break down an algorithm into functions.

- In a flowchart, function call shown as rectangle with vertical bars on each side
  - Function name written in the symbol
  - Typically draw separate flow chart for each function in the program
    - End terminal symbol usually reads `Return`



Hierarchy Chart

Hierarchy chart: depicts relationship between functions

- AKA structure chart
- Box for each function in the program, lines connection boxes illustrate the functions called by each function
- Does not show steps taken inside function
- In general, hierarchy corresponds to level from most abstract to most detailed of the function decomposition in the program

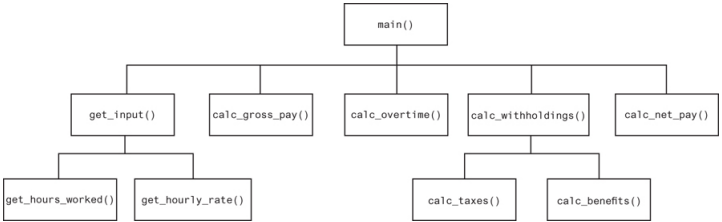


Figure 3: A hierarchy chart (Gaddis, 2021, pg.251)

EAST TEXAS A&M

Summary

- A function is a group of statements that exist within a program for the purpose of performing a specific task
- Functions are essential to good application development, benefits include:
  - Simpler code
  - Code reuse
  - Better testing and debugging
  - Faster development
  - Easier facilitation of teamwork
- Functions can be void functions, that return no value, or value-returning functions.
  - Likewise functions can take 0, 1 or many parameters as input
- Need 3 things to define and use a function
  1. Need to choose a meaningful name to call the function
  2. Need to determine input parameters (if any) for the function
  3. Need to determine if function returns a result, and if so what values it returns



EAST TEXAS A&M

References

- Developer, W. (2021). *Functions and why use them*. Retrieved December 4, 2021, from https://seattlewebsitedevelopers.medium.com/functions-and-why-use-them-74cd32fc2d72
- Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.
- Pacheco, T. (2023). *Best practices to write readable and maintainable code: Choosing meaningful names*. Retrieved February 14, 2023, from https://dev.to/pacheco/how-do-you-name-things-3jae
- Peppoloni, L. (2018). *The simplest one rule to write good functions*. Retrieved June 26, 2018, from https://medium.com/@l.peppoloni/the-simplest-one-rule-to-write-good-functions-61288249f077



EAST TEXAS A&M